

Fundamentals of Programming & Computer Science CS 15-112

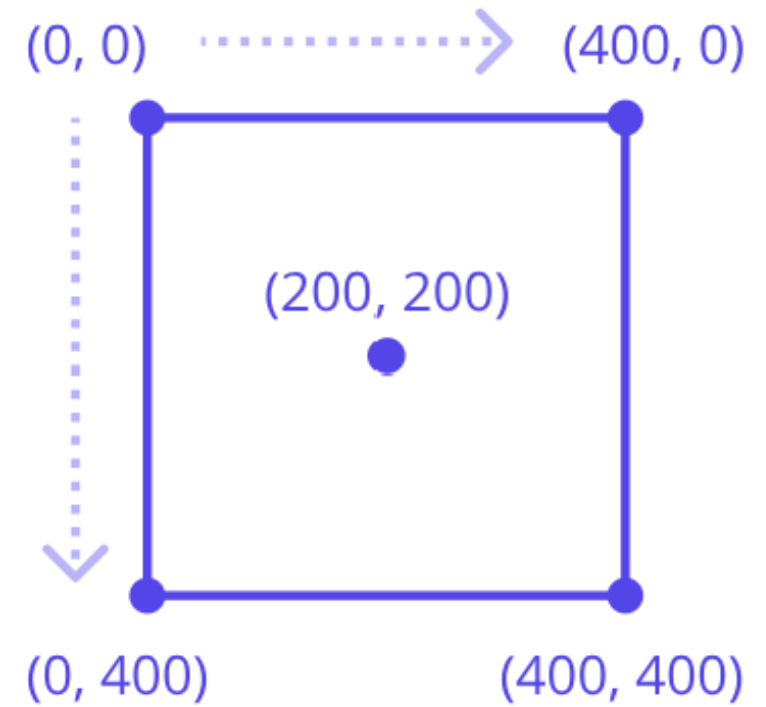
Graphics

Dr. Hend Gedawy

CMU Graphics Library

- CMU Graphics is a graphics package built at CMU for this course.
- On Cs Academy you will find
 - installation guide
 - Nice guided exercises
 - Inspector Tool
- Documentation:
 - <https://pypi.org/project/cmu-graphics/>
 - <https://academy.cs.cmu.edu/docs>
- Download CMU Graphics: <https://academy.cs.cmu.edu/desktop>
- CMU Graphics Reference Sheet
 - <https://academy.cs.cmu.edu/teacher-portal#resources>
 - https://drive.google.com/file/d/156zm1AOMu7XmC_BtPxcgZmgg2l9SxTQT/view

Canvas

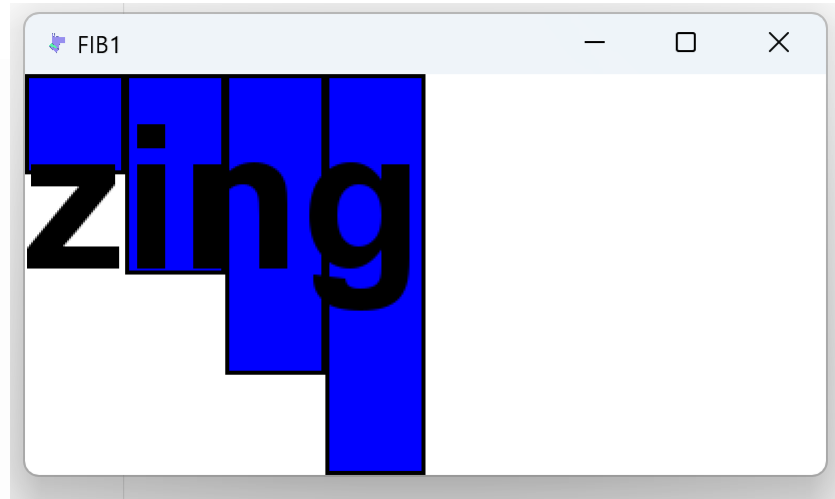


Graphics Practice

3. (10 points) **Fill-in-the-blanks:** Graphics

Fill in the blanks (A,B,C,D,E) so that this code draws the given image (shown below). Do not add any new lines. Just fill in the blanks. The image does not need to resize and you may assume the canvas is 400x200.

```
def FIB1(app):  
  
    w= 50 # with of first rect  
    h = 50 # height of the first rect  
  
    for i in range(4):  
  
        x= _____  
  
        y= _____  
  
        drawRect(_____, _____, _____, _____, fill='blue', border= 'black')  
  
        drawLabel(_____, _____, _____, align= _____, size=100, bold= True)
```



Graphics Practice

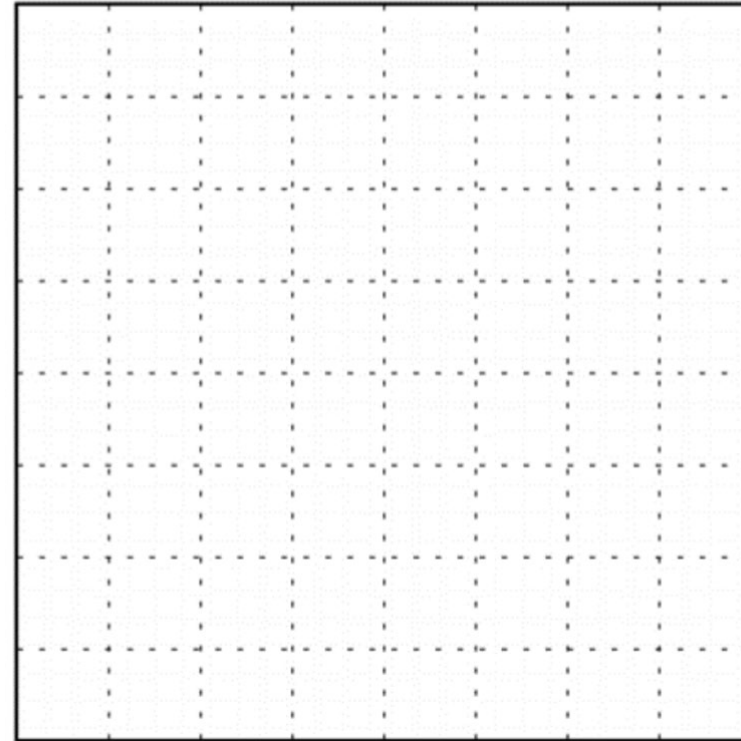
```
from cmu_graphics import *

def draw(app):
    x= 0
    y= 0
    d= 100

    count = 0
    for i in range(3):
        drawLabel(str(count), x+d//2, y+d//2, size=40)
        drawRect(x, y, d, d, fill=None, border='black')
        d+=50
        count+=1
        x+=0.5*d
        y+=0.5*d

def redrawAll(app):
    draw(app)

runApp()
```



Each square in the canvas grid is 50 x 50

Graphics Practice

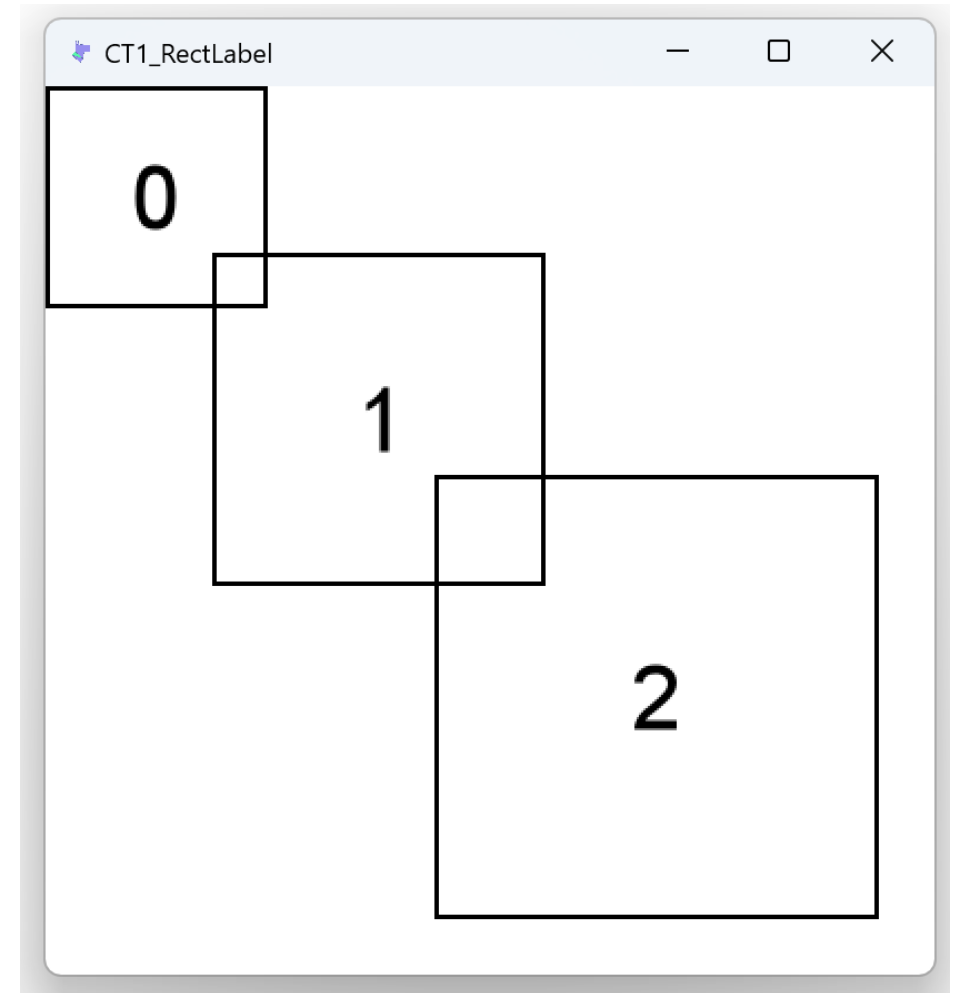
```
from cmu_graphics import *

def draw(app):
    x= 0
    y= 0
    d= 100

    count = 0
    for i in range(3):
        drawLabel(str(count), x+d//2, y+d//2, size=40)
        drawRect(x, y, d, d, fill=None, border='black')
        d+=50
        count+=1
        x+=0.5*d
        y+=0.5*d

    def redrawAll(app):
        draw(app)

runApp()
```



Last Lecture

- CMU graphics & CS Academy – overview
- Canvas
- Syntax
 - Rectangle
 - Attributes
 - Colors
 - Labels



CMU Graphics Reference Sheet

Function Names:

drawRect
 drawOval
 drawCircle
 drawRegularPolygon
 drawPolygon
 drawArc
 drawStar
 drawLine
 drawLabel

SHAPE PARAMETERS

SHAPE PARAMETERS

		fill	border	borderWidth	opacity	rotateAngle	dashes	align	visible	roundness	size	font	bold	italic	lineWidth	arrowStart	arrowEnd	
		default	'black'	None	2	100	0	False	'center'	True	None	12	arial	False	False	2	False	False
	Rect (left, top, width, height)	✓	✓	✓	✓	✓	✓	'left-top'	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗
	Oval (centerX, centerY, width, height)	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗
	Circle (centerX, centerY, radius)	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗
	RegularPolygon (centerX, centerY, radius, points)	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗
	Polygon (x1, y1, x2, y2, x3, y3, ...)	✓	✓	✓	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗
	Arc (centerX, centerY, width, height, startAngle, sweepAngle)	✓	✓	✓	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗
	Star (centerX, centerY, radius, points)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗
	Line (x1, y1, x2, y2)	✓	✗	✗	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗	✓	✓	✓	
text	Label (value, centerX, centerY)	✓	✓	✓	✓	✓	✗	✓	✓	✗	✓	✓	✓	✓	✗	✗	✗	

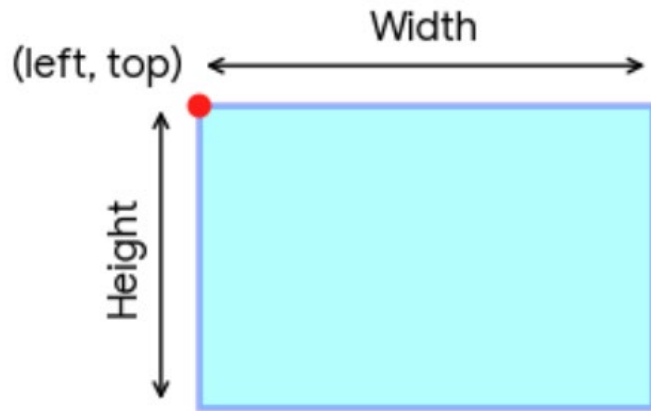
Position keywords: 'center', 'left', 'right', 'top', 'bottom', 'left-top', 'right-top', 'left-bottom', 'right-bottom'

✓ Shape has this property

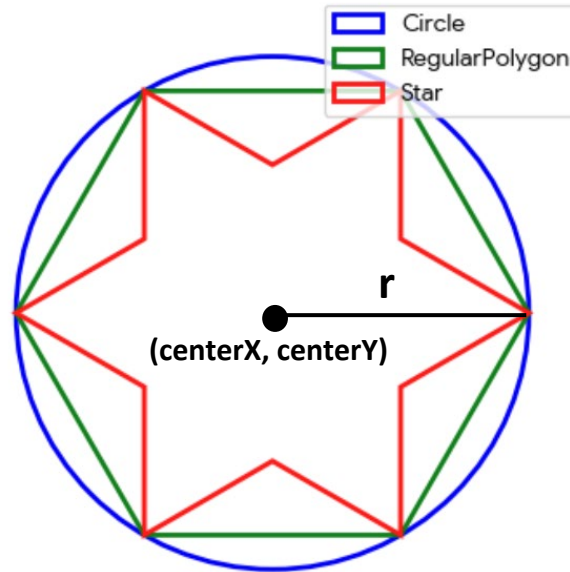
✗ Shape does not have this property

- <https://academy.cs.cmu.edu/desktop> >> Resources
- <https://academy.cs.cmu.edu/teacher-portal#resources>

Required Parameters Summary



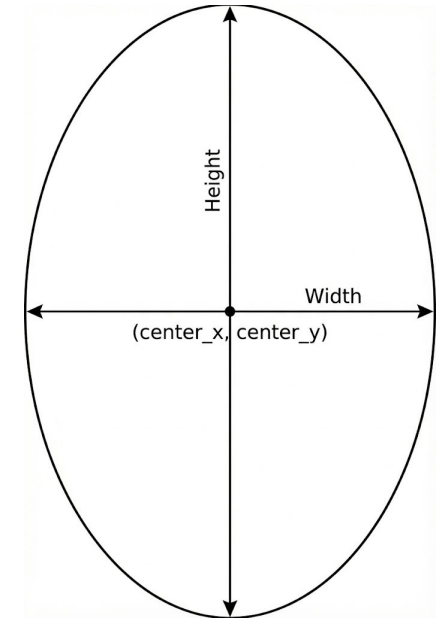
drawRect(left, top, width, height)



drawCircle(centerX, centerY, radius)

drawStar(centerX, centerY, radius, points)

drawRegularPolygon(centerX, centerY, radius, points)

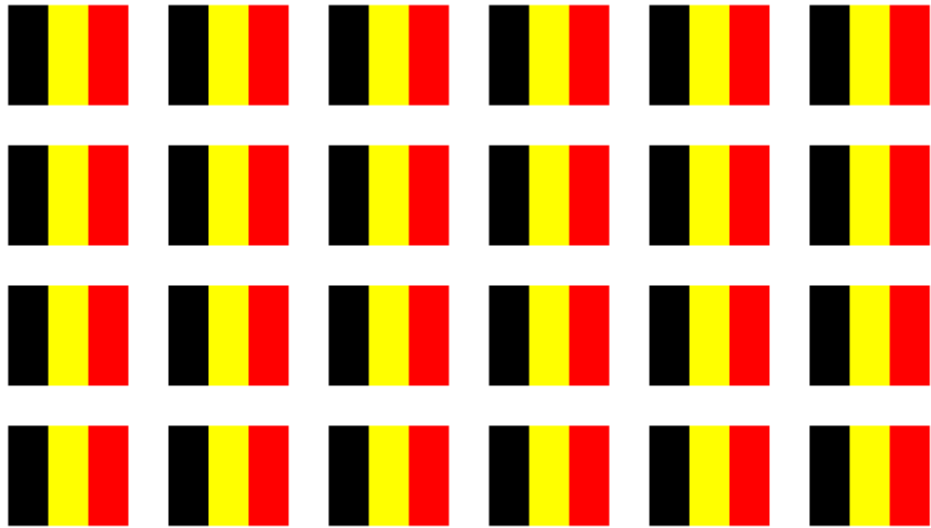


drawOval(centerX, centerY, width, height)

Required Parameters Summary

Shape	x, y (center)	x, y (top-left)	width	height	radius	points	text
Rectangle		□	□	□			
Oval	□		□	□			
Circle	□				□		
RegularPolygon	□				□	□	
Star	□				□	□	
Label	□						□

Belgian Flags



Belgian Flags

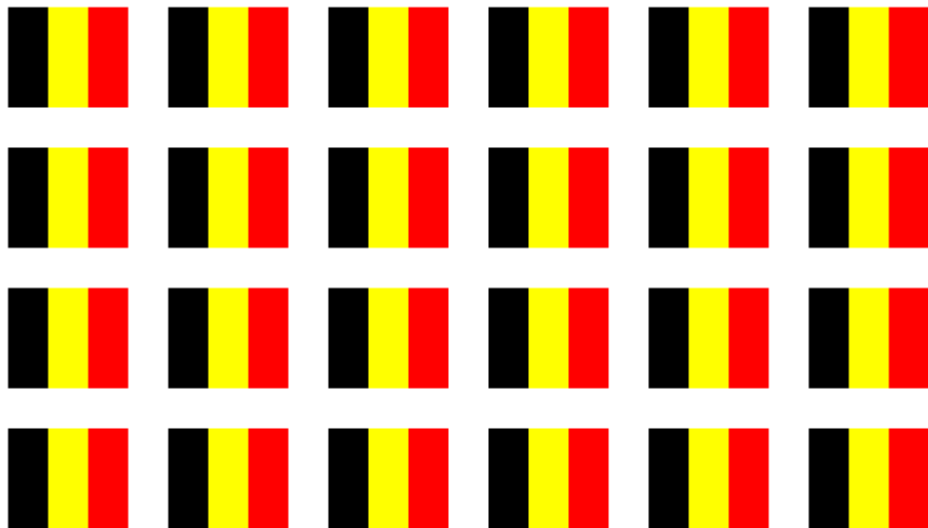
Write the **function** `drawBelgianFlag(x, y, w, h)` which draws the Belgian flag where (x,y) are the top-left corner coordinates of the flag and the flag dimensions are defined by w and h



Belgian Flags

Using the **drawBelgianFlag** helper function,
draw the following grid of Belgian flags (4
rows, 6 columns) ...

The flags are 10 pixels apart



Graphics Practice

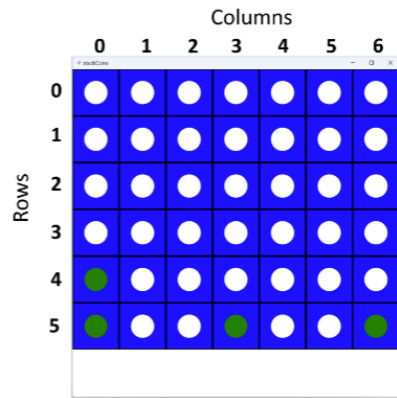
7. StackCoins Board Game

(a) (10 points) Fill-in The Blanks: Graphics

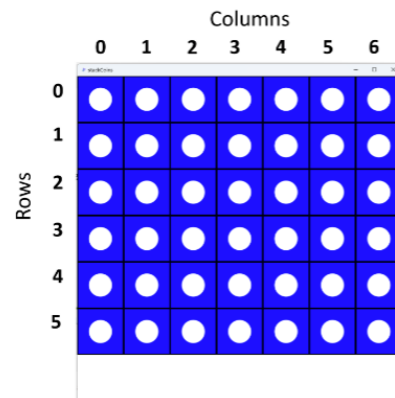
Fill in the code for the function `drawStackCoins(app, coinCells)`, which draws a 6x7 board game, with a total of 42 cells. The function takes as an input the string `coinCells` that specify the cells at which green coins are currently inserted. Each cell is represented as two numbers that map to rowID 0,1,2,...,5 and columnID 0,1,2,...,6; where cell(00) is the corner left top cell in the board.

For example, `drawStackCoins(app, "02,25")` draws two green coins/circles; one at cell 02 (i.e. row0 column2) and the other at cell 25 (i.e. row2 column5).

Consider the following examples (note that the printed row and column numbers are there to help your understanding, they are not present in the actual game):



(a) `drawStackCoins(app, "50,40,53,56")`



(b) `drawStackCoins(app, "")`

You can assume that `app.width==app.height`. The cells should be drawn to span the canvas width. Each cell should be drawn as a blue square with a black border and a circle centered in the square. The color of the circle depends on whether the cell is empty ('white') or has a ('green') coin. Your code should be able to handle window resize. You can assume that `cmu_graphics` is already imported and there is a function `redrawAll(app)` that will call your function.

```
def drawStackCoins(app, coinCells):  
    numRows = 6  
    numCols = 7  
    sideL = app.width // numCols
```

```
    for r in range(numRows):  
        for c in range(numCols):
```

```
            x = _____
```

```
            y = _____
```

```
            drawRect(x, y, _____)
```

```
            if _____ in _____:
```

```
                circleColor = 'green'
```

```
            else:
```

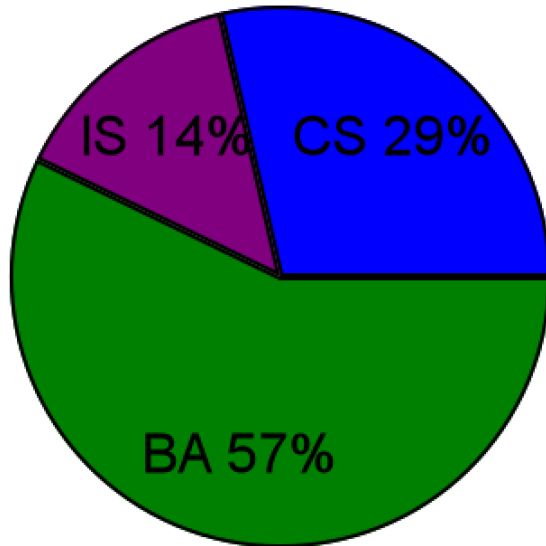
```
                circleColor = 'white'
```

```
            drawCircle(_____)
```


Draw Chart

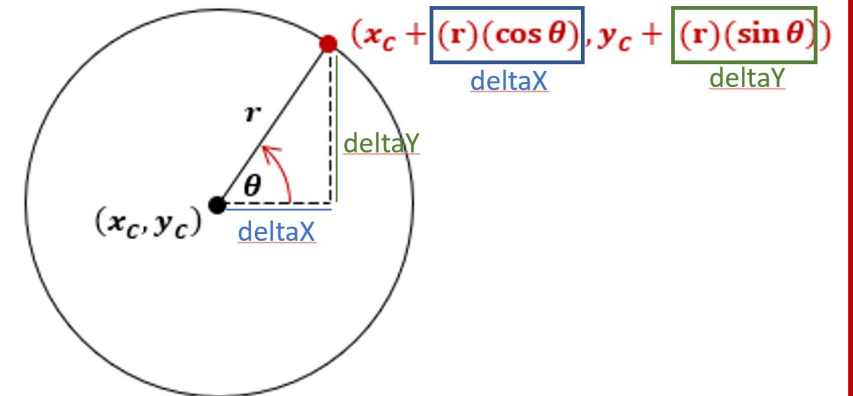
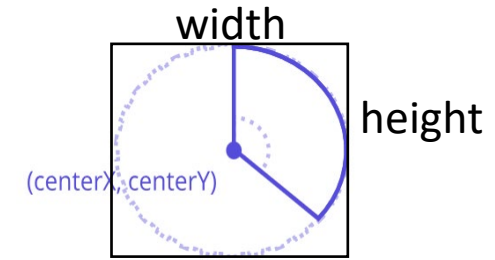
drawChart(s, total) takes a comma separated string (s) of value-label pairs and the sum of all values... And draws a pie chart. Your chart should alternate between three colors.

E.g. **drawChart(app, "10 CS,5 IS,20 BA", 35)**



Finding angle given percentage
Angle = (value/total) × 360

```
drawArc(centerX, centerY, width, height,  
startAngle, sweepAngle, fill='black')
```



Circle on Cartesian coordinates:

Its center: (x_c, y_c)

Its radius: r

Its angle: θ

Points on the circle: $(x_c + (r)(\cos \theta), y_c + (r)(\sin \theta))$

Graphics Practice

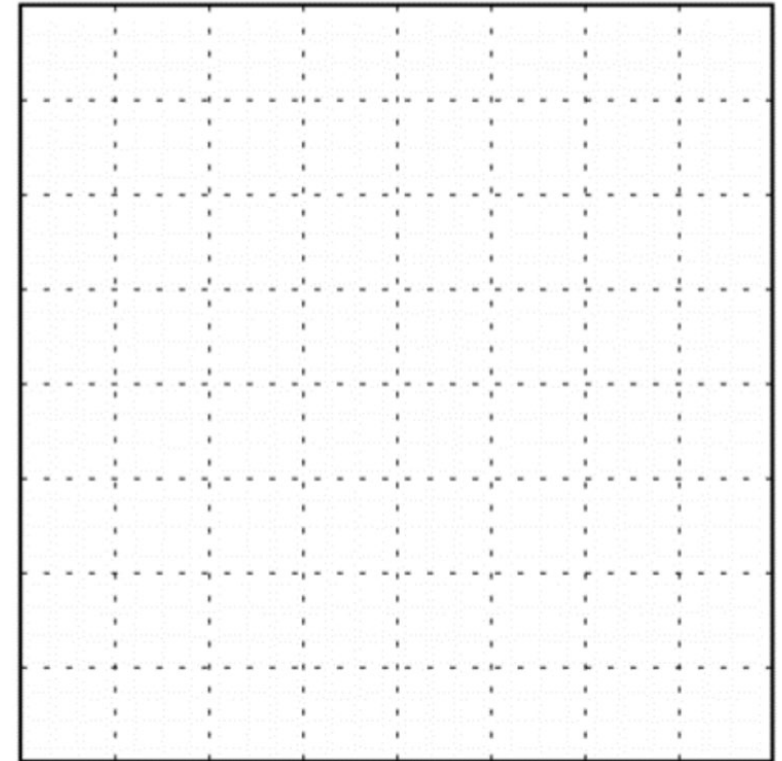
3. (10 points) Code Tracing: Graphics!

Note: Use a pencil when solving this problem. Trust us.

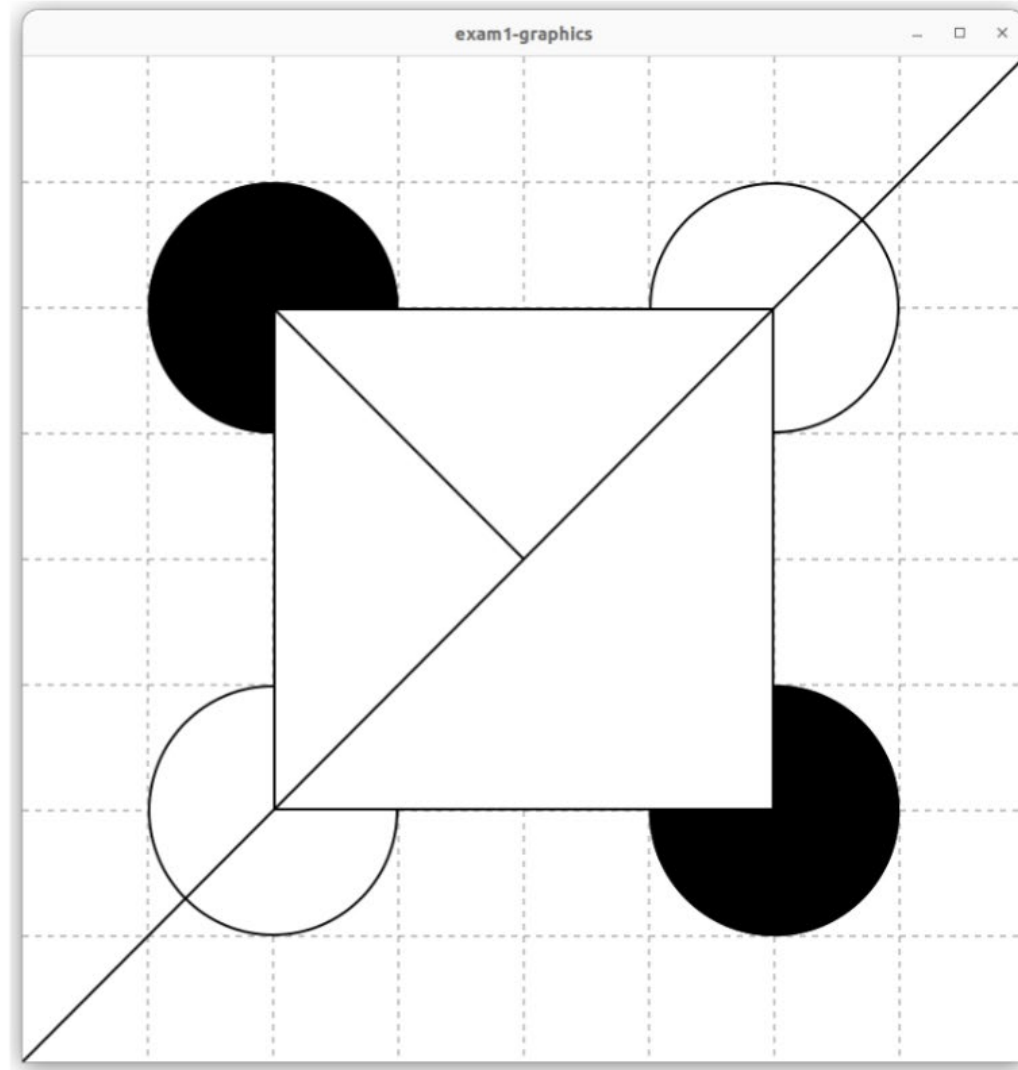
Given that the box below is your canvas, with a width and height of 400 each, draw what the following code will display. You can assume that this is called within the appropriate graphics helper code.

Hint: Each of the small boxes on the canvas is 50x50 pixels.

```
def redrawAll(app):  
    step = app.width//4  
    for i in range(2):  
        for j in range(2):  
            color = "white" if (i + j) % 2 == 1 else "black"  
            drawCircle(step + 2 * step * i, step + 2 * step * j, step / 2,  
                        border="black", fill=color)  
    drawRect(step, step, 2 * step, 2 * step, border="black", fill="white")  
    drawLine(0, app.height, app.width, 0)  
    drawLine(step, step, 2 * step, 2 * step)
```



Solution:



Grading:

- blah